



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Өгөгдлийн бүтэцлэлт. I хэсэг

Лекц 7

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Өгөгдлийн төрөл
- Төрлийн систем
- Скалар төрөл
- Нийлмэл төрөл
- Эквивалент байдал
- Нийцтэй байдал
- Төрөл хувиргалт



PL бүр өгөгдлийг зохион байгуулах бүтэц, механизмтай байдаг.

Жнь: Илэрхийлэл болон санах ойн объектуудын төрөл

Төрлийн систем: *Дээд түвшний хэл нь* тухайн шийдэх асуудлын бүтцийг илүү хялбар зохион байгуулдаг комплекс, бүтээцлэгдсэн өгөгдөлтэй (Физик машинд бол энгийн бит цуваа байдаг. Ихэнх ассемблэр хэл төрөлгүй).

**Анхдагч төрлүүд ба шинэ төрөл бүтээх механизм;
Эквивалент чанар; Нийцтэй байдал; Төрөл хувиргалт**

- Зохиомжийн түвшинд: **Conceptual organization**
 - Консептын зохион байгуулалтанд өгөх дэмжлэх
 - Үйлдлийн контекстийг удирдах; $a+b$: нэмэх vs тэмдэг мөр залгах;
 - PL хэрэгжилт нь энэ контекст мэдээллийг ашигладаг
- Програмын түвшинд: **Support for correctness**
 - Алдаагүй байдалд өгөх дэмжлэг
 - Хүчинтэй үйлдлийн шаардлага: Тэмдэгт + бичлэг, Set дээр логарифм авах
 - Код хялбар, ойлгомжтой байх; Хөгжүүлэгч гажгийг (bug) эрт мэдэх
- Хөрвүүлэлтийн түвшинд: **Support for the implementation**
 - Хэрэгжилтэнд өгөх дэмжлэг
 - Оновчлол: Компилятор зарим үйлдэл боломжгүй гэдгийг мэддэг
 - Ил болон далд зарлалттай төрлийн алинд нь ч ажиллана

ТОД 8.1 (Дата төрөл): Нэгэн төрлийн (homogeneous) утгын **үр ашигтайгаар байгуулагддаг** цуглуулга бөгөөд эдгээр утгуудыг зохицуулах **үр ашигтай** үйлдлүүдтэй байна.

- Төрөл бол утгуудын олонлог: бүхэл тоо, интегралын интервал шиг.
- Нэгэн төрлийн: утгууд нь ижил бүтэцтэй байх
- “Үр ашигтайгаар байгуулагддаг” утга
 - Математик биш алгоритм бичих хэлний хувьд төгсгөлөг (бичих, нэрлэх) байх
 - Бодит тоо: төгсгөлгүй тул ямар ч алгоритмаар олж авах боломжгүй
- Утга нь үйлдэлтэй: Бүхэл тоог $+$, $-$, $*$, $/$. Мөн $\%$, зэрэг дэвшүүлэх
- “Үр ашигтай” үйлдэл
 - Утгын төлөөлөгч дээр үйлдлийг гүйцэтгэх механик, бодит арга зам байх

- Комплекс асуудал бүрийн шийдэл үргэлж **концептийн** бүтэцтэй байдаг
 - Төрөл олон байгаа нь концептын класс бүрт сонгох боломж олгодог.
 - Жнь, зочид буудлын программ нь клиент, огноо, үнэ, өрөө зэрэг консепттой. Консепт бүрийг өөр өөр төрөл, үйлдлүүдтэй байхаар тодорхойлж болно
- Шинэ төрөл тодорхойлно; Ижил утгаар илэрхийлэгдэж байсан ч ялгаж чадна.
 - Жнь, өрөө болон үнийг адилхан бүхэл тоон интервалаар илэрхийлж болох ч ялгаатай төрлөөр дүрсэлж концептыг тусгаарладаг.
- Ялгаатай төрлийн хэрэглээг дизайн ба документ хэрэгсэл гэж үзэж болно.
 - Програмыг уншиж байхдаа, "room" болон "price" төрлийн хувьсагчид нь өөр үүрэгтэйг зарлалтаас нь ойлгож болно. Энэ утгаараа төрөл нь комменттэй төстэй гэж болно:

Тодорхой программчалын хэлэнд тодорхой нөхцлийн дор **үр ашигтайгаар шалгагдах боломжтой** коментуудтай ажилладаг гэсэн үг

- Төрөл шалгах (type-checking):
 - `int+record/struct`; объект (функц/процедур биш) дуудалтыг хориглох (хаяг!)
 - `X:=exp` командын хувьд `exp` болон `x`-ийн (зарласан) төрлүүд ижил байх.
- Төрлийн шаардлагууд нь семантик (дизайны) алдаанаас сэргийлнэ
 - Компилятор статик семантик шалгах үе шат
 - Төрөл (комментоос ялгаатай нь) шалгах нь объектын буруу хэрэглээг мэдрэх
- Логик алдаанууд бичих дүрмийн зөрчлөөр далдлагддаг: `int+string`
 - Төрөл шалгалт алдаагүй байдлын хамгийн доод түвшин боловч хөгжүүлэлтийн шатанд мэдэгдэхүйц давуу талтай (Физикчид томьёон дээр ажиллахаас өмнө хэмжилтээ хянадаг)
- Гэхдээ төрөл шалгалт нь маш хязгаарлагдмал:
 - C заагчийн чөлөөт хэрэглээ (дурын арифметик заагч), Java-д шаардахгүй
 - Олон хэлбэржилт (Polymorphism), Хуурамч лавлагаа (Dangling references)

ДАТА ТӨРӨЛ Support for the implementation

- Төрөл нь хийсвэр машинд шаардлагатай мэдээллийн чухал эх сурвалж
 - Объектын санах ой: `int` – нэг үг, логик утга – нэг байт, n дараалалд – n үг
 - Хэрэв төрлүүд статик бол энэ мэдээллийг компиляцад ашиглаж болох бөгөөд биелэлтийн явцад өөрчлөгдөхгүй.
 - Тиймээс объект руу хандах боломжийг оновчтой болгож чадна.

```
struct Professor{  
    char Name[20];  
    int Course_code;  
}
```

- Professor төрөлтэй `p` хувьсагч
 - `P` нь санах ойд хуваарилагдсан `p` гэсэн `P.Name`, `P.Course_code` талбаруудад оффсет хаягийн тусламжтойгаар шууд хандана

Төрлийн систем

Type system: Тухайн хэлний төрлийг удирдах мэдээлэл, дүрмийн иж бүрдэл

1. Хэлний урьдчилан тодорхойлсон төрлүүд.
2. Шинэ төрөл тодорхойлох механизмууд.
3. Төрлүүдийг хянах механизмууд:
 - **Эквивалентийн дүрэм:** Ялгаатай формал төрлүүд хоорондоо тохирох
 - **Нийцтэй байдлын (Compatibility) дүрэм:** Утгыг өөр төрлөөр хэрэглэх
 - **Төрөл гаргалгааны (inference) дүрэм, арга техник:** Төрлийг бүрдэл хэсгүүдэд нь суурилан комплекс илэрхийллээр тодорхойлох
4. Статик эсвэл динамикаар шалгадаг эсэх (эсвэл нэгээр нь) тодорхойлолт.
 - Тухайн хэлний төрөл хоорондын ялгааг ямар ч программ зөрчих боломжгүй бол төрлийн систем нь **аюулгүй (type safe)** байна.

- PL-ний төрлүүдийг тухайн төрлийн утгыг хэрхэн өөрчилж болох, эдгээр утгуудад тохирох синтакс нэгжийн үүднээс ангилж болно.
 - **Тэмдэглэгдэх боломжтой (Donatable):** Нэртэй холбож болох.
 - **Илэрхийлэгдэх боломжтой (Expressible):** Комплекс илэрхийллийн үр дүн
 - **Хадгалагдах боломжтой (Storable):** Хувьсагчид хадгалах боломжтой
- Бүхэл тоон төрлийн утгуудын хувьд дээрх 3 биелнэ
- Функциональ утгууд
 - Нийтлэг императив хэл үр дүн буцаах илэрхийлэлгүй expressible биш
 - Хувьсагчид функц оноох чадахгүй тул ерөнхийдөө storable биш.
- Гэхдээ функциональ хэлнүүд бусад парадигмуудаас ялгаатай,
 - Функциональ хэлэнд (Scheme, ML, Haskell гэх мэт) функциональ утгууд нь хоёуланг нь donatable, expressible, зарим хэлэнд нь storable байдаг.
 - Python-д ямар ч утга donatable, expressible; нэрийг run-time онооно.

- Үйлдлийг зөв төрөлтэй операндууд дээр хэрэглэж байгаа эсэх.
- **Динамик**: Компилятор үйлдэл бүрийн өмнө шалгах кодыг үүсгэдэг байж болох ч үндсэн үйлдэлтэй нь төрөл шалгалт холилдсон байх тул ашиггүй
 - Мөн алдаа нь зөвхөн эцсийн хэрэглэгч дээр гарах боломжтой байж болно.
- **Статик**: Бүрэн статик бол ажиллах үед шалгахгүй тул биелэлт ашигтай
 1. Динамик загвараас илүү комплекс, ялангуяа статик шалгалтын хамт төрлийн аюулгүй байдлыг хангахыг хүсч байвал.
 2. Компиляц нь илүү урт ба комплекс болох ч цөөн удаа хийгддэг, гол нь туршилт болон дибаг үе шатуудыг богиносгодог тул хүлээн зөвшөөрөгддөг.
 3. Бодит байдал дээр статик төрлүүд нь ажиллах үед төрлийн алдаа үүсгэхээргүй программыг алдаатай гэж тодорхойлж болно.

```
int x;  
if (0==1) x = "pippo";  
else x = 3+4;
```

Утга нь өөр утгуудын нийлмэл байдаггүй бол
скаляр (эсвэл энгийн) төрөл байна.

- **type** newtype = *expression*;
 - newtype - төрлийн нэр, **expression** - бүтэц
 - C хэл дээр: **typedef** *expression* newtype;
-
- | | | |
|-------------|---------------|-----------------|
| • Boolean | • Fixed Point | • Enumerations |
| • Character | • Complex | • Intervals |
| • Integers | • Unit | • Ordered Types |
| • Reals | | |

Бусад төрлийг тохиромжтой *байгуулагч (constructor)* бий болгодог тул скаляр бус төрлийг **нийлмэл (composite)** гэж нэрлэдэг.

- **Record** (or **struct**), янз бүрийн төрлийн утгуудын эрэмбэтэй цуглуулга.
- **Union** and **tagged union**, бусад төрлийн нэгдлээс үүссэн цуглуулга.
- **Array** (or **vector**), ижил төрлийн утгуудын эрэмбэтэй цуглуулга.
- **Set**: суурь төрлийн дэд олонлогууд, ерөнхийдөө дарааллын төрлүүд.
- **Pointer**: өөр төрлийн өгөгдөлд хандахыг зөвшөөрдөг *l*-утгууд.
- **Sequence**: төгсгөлөг утгуудын дараалал (дурын төрлийн).
- **Recursive**: тогтмол болон байгуулагч ашиглан рекурсээр тодорхойлсон төрлүүд; Жнь: жагсаалт, мод ...
- **+ Functions**:

Төрлийн эквивалент байдал

Хоёр төрөл эквивалент байх: Хэрэв нэг төрлийнх нь илэрхийлэл эсвэл утга бүр нь нөгөөгийнх нь илэрхийлэл эсвэл утга байна, мөн эсрэгээрээ.

```
type newtype = expression;
```

- Хэлнээс хамаарч төрлийн эквивалентийн хоёр өөр дүрэмтэй:
 - Төрлийн тодорхойлолт нь *шууд биш (opaque)* байх: **Нэрээр эквивалент**
 - Төрлийн тодорхойлолт нь *шууд (transparent)* байх: **Бүтцийн эквивалент**

ТОД 8.2 (Нэрийн эквивалент) Хоёр төрөл нь зөвхөн ижил нэртэй байх үед нэрээрээ эквивалент байна (төрөл нь зөвхөн өөртэйгээ эквивалент).

- Нэрийн эквивалент нь хэт хязгаарлагдмал байдаг.

```
type T1 = 1..10;  
type T2 = 1..10;  
type T3 = int;  
type T4 = int;
```

*4н тусдаа төрөл,
эквивалент байхгүй.*

- Зарим хэл нэрийн **сул (weak) эквивалент** дүрэмтэй.
 - Төрлийн нэрийг өөрчлөх (rename) нь шинэ төрөл үүсгэхгүй, зөвхөн ижил нэртэй алиасыг болдог.
 - Дээрх жишээнд нэрийн сул эквивалентаар T3 ба T4 төрлүүд нь эквивалент, T1 ба T2 төрөл нь ялгаатай хэвээр байна.

ТОД 8.3 (Бүтцийн эквивалент) Дараах нөхцлүүдийг хангасан эквивалент (хамгийн бага) хамаарал.

- Төрлийн нэр өөртэйгээ нь эквивалент;
- Хэрэв T төрөл нь $\text{type } T = \text{expression}$ бол expression -тэй эквивалент байна;
- Хэрэв хоёр төрлийг эквалент төрлүүд дээр ижил төрлийн байгуулагч хэрэглэн бүтээсэн бол хоёр төрөл нь эквивалент байна.

```
type T1 = int;
type T2 = char;
type T3 = struct{
    T1 a;
    T2 b;
}
type T4 = struct{
    int a;
    char b;
}
```

Т3 ба Т4
бүтцийн
эквивалент

```
type S = struct{
    int a;
    int b;
}
type T = struct{
    int n;
    int m;
}
type U = struct{
    int m;
    int n;
}
```

Нэрс, талбарын
дараалал өөр

```
type R1 = struct{
    int a;
    R2 p;
}
type R2 = struct{
    int a;
    R1 p;
}
```

төрөл шалгагч нь
харилцан рекурсийг
шийдвэрлэх
боломжгүй

Төрлийн нийцтэй байдал (Compatibility)

ТОД 8.4 (*Нийцтэй байдал*): Хэрэв S төрлийн утга хэрэглэх аливаа контекстэд T төрлийн утгыг зөвшөөрдөг бол T нь S -д тохирно гэнэ.

- Олон хэлний хувьд утга олголт, параметр дамжуулалт зэргийн алдаагүй байдлыг хянадаг дүрэм (Эквивалент дүрэм биш).
 - `int` ба `float` (эсрэгээрээ биш) хоорондоо нийцтэй (гэхдээ ML, Java-д үгүй).
- Ерөнхий тохиолдолд авч үзвэл **нийцтэй байдлын харьцаа** нь эквивалент байдлаас илүү хэл хооронд асар их **ялгаатай**

1. T ба S төрлүүд нь **эквивалент**; хамгийн хязгаарлагдмал нийцтэй байдал
2. T-ийн утгууд нь S-ийн утгуудын **дэд олонлог** болдог
3. S-ийн утгууд дээрх **бүх үйлдлийг** T-ийн утгууд дээр **зөвшөөрдөг**. Хэд хэдэн өөр хувилбараар татгалзаж болно.
 - *Structural compatibility*
 - *Нугасан төрөлтэй (duck typing)*
 - *Subtype харьцаа*
4. T-ийн утгууд нь S-ийн утгуудтай **каноник хэлбэрээр** харгалздаг.
 - (2) биш, хэрэгжилтийн хувьд хоёр төрлийн утгууд ялгаатай.
5. T-ийн утгууд S-ийн **зарим утгатай** харгалздаг.
 - Хувиргалтыг дурын байдлаар тодорхойлж нийцтэй болгоно. Жнь float-ыг int-тэй нийцтэй болгож хувиргана (rounding, truncation гэх мэт).

Хувиргалт (Conversion)

- Нийцтэй байдлын харьцааны ялгаатай боловсруулалтыг нэгдсэн байдлаар удирдах шаардлагаар **төрөл хувиргалт** хийдэг
- **Далд хувиргалт: албадлага (coercion)** эсвэл албадан хөрвүүлэх.
 - Дээд түвшний хэлэнд хувиргалтын ямар нэг заавар байхгүй үед хийсвэр машин нь хувиргалтыг нэмж оруулдаг;
 - Т ба S төрлүүд хоорондоо нийцтэй байхад S-ийн утга байх ёстой газар Т-ийн утгыг хэл нь зөвшөөрдөг бол компилятор ба/эсвэл хийсвэр машин Т ба S хооронд төрөл хувиргалтыг нэмж оруулдаг.
 - Синтаксийн үүднээс албадлага нь нийцтэй байдлын тайлбараас өөр ач холбогдолтой байдаггүй.
- **Ил хувиргалт: cast.** Хувиргалтыг программын текстэнд бичиж өгсөн.

Хэрэгжилтийн хувьд албадлага нь нийцтэй байдлын ойлголтоос хамааран:

1. T нь S -тэй нийцдэг, ($T \subseteq S$), хадгалах дүрслэл ижил (ядаж T -ийн утгууд)
 - Албадлага нь синтаксийн асуудал хэвээр үлдэж, нэмэлт код үүсгэхгүй.
 2. T нь S -тэй нийцдэг ч T -ийн утгыг S -ийн утга болгон хувиргах ёстой.
 - Албадлага биелж хийсвэр машин хувиргалтыг нарийн хэрэгжүүлдэг. Жнь, `int` ба `float`-ын хувьд бүхэл тоон дүрслэлийг хөвөгч цэгийн болгох код нэмдэг.
 3. T ба S -ийн утгууд хоорондын дурын харгалзаан дээр суурилдаг.
 - Хийсвэр машин нь мөн л хувиргалтыг гүйцэтгэх кодыг нэмдэг.
 - T ба S нь ижил дүрслэлтэй, $S \subseteq T$ бол албадлага дүрслэлийг өөрчлөхгүй ч T -ийн утга S -г дагах эсэхийг динамикаар шалгана (ядаж хэл аюулгүй байх).
- ***Төрлийг хатуу шалгадаг хэлэнд (3) бага (нийцтэй байдал ховор).***
 - Си шиг хэлний төрлийн систем нь (by-pass) тул албадлагыг олон хийдэг.
 - `char`-аас `int`, урт `real`-ээс богино, урт `int`-ээс богино `real` руу гэх мэт

ХУВИРГАЛТ >> Ил (explicit) хувиргалт

- Ил хувиргалт нь нэг төрлийн утгыг нөгөөгий утгад хувиргах тэмдэглээ.
 - Синтакс тэмдэглэгээ (cast) байна, эсвэл албадлага ашиглана.

$S \quad s = (S) \quad t ;$

- Хэл нь яаж хэрэгжүүлэхийг мэддэг бол ил хувиргалтыг зөвшөөрнө.
 - Нийцтэй байдал бий болсон газар cast оруулж болох нь тодорхой.
 - Энэ тохиолдолд синтаксийн хувьд ашиггүй ч, документ зорилгоор заавал ашиглах ёстой
 - Нийцтэй байдал багатай хэл нь ерөнхийдөө олон ил хувиргалтыг шаарддаг
- Ерөнхийдөө орчин үеийн хэлнүүд албадлагаас cast-ийг илүүд үздэг.
 - Документын хувьд илүү ашигтай; тэдгээр нь гарч ирж буй синтакс контекстээс хамаардаггүй
 - Хамгийн чухал нь overloading, полиморфизмын үед илүү сайн ажилладаг



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА